

Reviewer Pack — Minimal Geometric Entropy of Boolean Functions Bounds Resolu...

Entry 0d77dc6e5a48 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 21:10:01 UTC

Minimal Geometric Entropy of Boolean Functions Bounds Resolution Proof Depth

Entry ID: 0d77dc6e5a48 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 21:10:01 UTC

1. Conjecture

Field A (mathematical branch): Classical Mechanics (Hamiltonian Dynamics) **Field B** (complexity object): Boolean Function Complexity: Tree-like Resolution Proof Depth

Statement:

For a given n -variables boolean function f , let $H(f)$ be its minimal geometric entropy under all possible Hamiltonians that preserve the Boolean algebra structure. Then, the resolution proof depth $t^*(f)$ of f is upper-bounded by $H(f)^2$.

Rationale (proposer's reasoning):

The minimal geometric entropy of a boolean function could provide insights into the difficulty of finding a proof for it. A lower bound on this entropy suggests that the function requires deeper proofs. This conjecture might expose a new connection between geometric entropy and complexity, potentially revealing non-intuitive relationships between these two fields.

Taxonomy category: cg_kw_andreev (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3dfbf3d1bce455c7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for all 30 seeds, $H(f)^2 \leq t(f)$ AND correlation coefficient ≥ 0.7 ; it is falsified if any seed fails $H(f)^2 > t(f)$ OR correlation coefficient < 0.5 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal geometric entropy" AND (Hamiltonian dynamics OR classical mechanics) - "Boolean function complexity" AND "resolution proof depth" AND (tree-like resolution OR Hamiltonian) - $H(f)$ IN BOOLEAN FUNCTION COMPLEXITY AND $t^*(f) \leq H(f)^2$

Top relevant hits considered: - [http://arxiv.org/abs/2508.18081v2] GWTC-4.0: Methods for Identifying and Characterizing Gravitational-wave Transients - [http://arxiv.org/abs/2604.05701v1] Measurement of the CKM angle γ in $B^\pm \rightarrow D(\rightarrow K_S^0 h'^+ h'^-) h^\pm$ dec - [http://arxiv.org/abs/2505.00274v1] Future Circular Collider Feasibility Study Report: Volume 2, Accelerators, Technical Infrastructure and Safety

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=241.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def hamiltonian_dynamics(f):
        n = int(math.log2(len(f)))
        H_f = 0
        for i in range(2**n):
            if f[i] == 1:
                H_f += 1
        return H_f / (2**n)

    def resolution_proof_depth(f):
        # Placeholder function to simulate the computation of resolution proof depth
        # This is a dummy implementation and should be replaced with actual logic
        n = int(math.log2(len(f)))
        return n * 2

    instances_tested = 0
    n_max = 0
    total_H_f_squared = 0
    total_t_star = 0
```

```

for n in [5, 10, 15, 20, 30, 40]:
    f = generate_boolean_function(n)
    H_f = hamiltonian_dynamics(f)
    t_star = resolution_proof_depth(f)

    instances_tested += 1
    if n > n_max:
        n_max = n

    total_H_f_squared += H_f ** 2
    total_t_star += t_star

mean_H_f_squared = total_H_f_squared / instances_tested
mean_t_star = total_t_star / instances_tested

conjecture_holds = mean_H_f_squared <= mean_t_star
correlation_coefficient = (instances_tested * mean_H_f_squared * mean_t_star -
                           total_H_f_squared * total_t_star) / (
    math.sqrt((instances_tested * sum(H_f ** 4 for H_f in [hamiltonian_dynamics(f) for f in generate_boolean_functions(n_max)])
              (instances_tested * sum(t_star ** 4 for t_star in [resolution_proof_depth(f) for f in generate_boolean_functions(n_max)])

return {
    "metric_name": "correlation_coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot verify the conjecture for all seeds as required by the pre-registered support condition. | next: Re-run the test with a longer timeout or an alternative method to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16080
2	preregistration	ollama_remote	glm4:latest	0	0	9199
3	novelty	ollama_remote	glm4:latest	0	0	14047
4	novelty	ollama_remote	glm4:latest	0	0	21436
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15021
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11642
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	41620
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	55236
9	judge	ollama_remote	glm4:latest	0	0	20101

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 204383 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/0d77dc6e5a48.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0d77dc6e5a48.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/0d77dc6e5a48.tar.gz` (if generated)